```
# CELL 0 - Environment check and installs
!nvidia-smi || echo "No GPU detected (OK, will use CPU)."

# Install packages
import sys
!pip install -q ultralytics==8.1.23  # or latest stable if you prefer
!pip install -q ensemble-boxes pycocotools tqdm

import os
if not os.path.exists('/content/yolov5'):
    !git clone -q https://github.com/ultralytics/yolov5.git /content/yolov5
    %cd /content/yolov5
    !pip install -q -r requirements.txt
    %cd -
print("Install step done.")
```

```
/bin/bash: line 1: nvidia-smi: command not found
No GPU detected (OK, will use CPU).
Install step done.
```

```
# CELL 1 - show folder tree & image counts under /content/datasets
import os, glob
ROOT = '/content/datasets'
if not os.path.exists(ROOT):
    print("No /content/datasets folder exists. Upload your dataset to /content/datasets or mount Drive.")
else:
    print("Top-level items in /content/datasets:")
    for name in sorted(os.listdir(ROOT)):
        path = os.path.join(ROOT, name)
        if os.path.isdir(path):
            imgs = glob.glob(os.path.join(path, '**', '*.*'), recursive=True)
            img_count = sum(1 for f in imgs if f.lower().endswith(('.jpg','.jpeg','.png','.bmp')))
            print(f" - {name:30s}  | images (recursively): {img_count}")
        else:
            print(" - (file) ", name)
print("\nIf your images are not under /content/datasets, upload them now (zip is easiest).")
```

```
Top-level items in /content/datasets:
 - (file)  LICENSE.txt
 - (file)  african-wildlife.yaml
 - (file)  african-wildlife.zip
 - african_small                  | images (recursively): 250
 - images                         | images (recursively): 1504
 - labels                         | images (recursively): 0

If your images are not under /content/datasets, upload them now (zip is easiest).
```

```
# CELL 2 - Build subset into /content/datasets/african_small (preserve classes if present)
import os, glob, random, shutil
ROOT = '/content/datasets'
SUBSET = os.path.join(ROOT, 'african_small')
TRAIN_OUT = os.path.join(SUBSET, 'images', 'train')
VAL_OUT = os.path.join(SUBSET, 'images', 'val')

MAX_IMAGES = 250        # cap total images (change lower if slow)
MAX_PER_CLASS = 200     # cap per class if classes exist
RSEED = 42
random.seed(RSEED)

# collect candidate image folders and classes
def find_class_folders(root):
    classes = []
    for entry in sorted(os.listdir(root)):
        full = os.path.join(root, entry)
        if os.path.isdir(full):
            # check if folder directly contains images
            imgs = [f for f in os.listdir(full) if f.lower().endswith(('.jpg','.jpeg','.png','.bmp'))]
            if imgs:
                classes.append(entry)
    return classes

classes = find_class_folders(ROOT)
# If no direct class folders at top, try deeper one level
if not classes:
    for d in sorted(os.listdir(ROOT)):
        p = os.path.join(ROOT, d)
        if os.path.isdir(p):
            inner = find_class_folders(p)
            if inner:
                classes = inner
```

```
                # remap ROOT to that folder for searching images
                ROOT = p
                print("Using nested folder", p, "as dataset root")
                break

print("Detected classes (if any):", classes)

# Clear & create dest
shutil.rmtree(SUBSET, ignore_errors=True)
os.makedirs(TRAIN_OUT, exist_ok=True)
os.makedirs(VAL_OUT, exist_ok=True)

# If class folders exist under ROOT, preserve them
total_copied = 0
if classes:
    print("Preserving class subfolders. Copying up to MAX_PER_CLASS per class until MAX_IMAGES total reached.")
    for cls in classes:
        src_dir = os.path.join(ROOT, cls)
        imgs = sorted([os.path.join(src_dir,f) for f in os.listdir(src_dir) if f.lower().endswith(('.jpg','.jpeg','.png','.
        if not imgs:
            continue
        # sample up to MAX_PER_CLASS
        selection = imgs[:MAX_PER_CLASS]
        # split 80/20
        split = int(0.8 * len(selection))
        train_imgs = selection[:split]
        val_imgs = selection[split:]
        # copy into train/val with class subfolder
        for s in train_imgs:
            dst_dir = os.path.join(TRAIN_OUT, cls); os.makedirs(dst_dir, exist_ok=True)
            shutil.copy2(s, os.path.join(dst_dir, os.path.basename(s))); total_copied += 1
            if total_copied >= MAX_IMAGES: break
        if total_copied >= MAX_IMAGES: break
        for s in val_imgs:
            dst_dir = os.path.join(VAL_OUT, cls); os.makedirs(dst_dir, exist_ok=True)
            shutil.copy2(s, os.path.join(dst_dir, os.path.basename(s))); total_copied += 1
            if total_copied >= MAX_IMAGES: break
        if total_copied >= MAX_IMAGES: break
else:
    # flat copy: find images anywhere under original root (excluding african_small itself)
    candidates = []
    for root, dirs, files in os.walk(ROOT):
        if 'african_small' in root.split(os.sep): continue
        for f in files:
            if f.lower().endswith(('.jpg','.jpeg','.png','.bmp')):
                candidates.append(os.path.join(root,f))
    candidates = sorted(set(candidates))
    if not candidates:
        raise RuntimeError(f"No images found under {ROOT}. Upload images to /content/datasets and re-run.")
    # sample up to MAX_IMAGES
    if len(candidates) > MAX_IMAGES:
        candidates = random.sample(candidates, MAX_IMAGES)
    split = int(0.8 * len(candidates))
    train_imgs = candidates[:split]; val_imgs = candidates[split:]
    for s in train_imgs:
        shutil.copy2(s, os.path.join(TRAIN_OUT, os.path.basename(s))); total_copied += 1
    for s in val_imgs:
        shutil.copy2(s, os.path.join(VAL_OUT, os.path.basename(s))); total_copied += 1

print(f"Copied total {total_copied} images into subset {SUBSET}")
print("Train sample files:", next(os.walk(TRAIN_OUT))[1] if os.path.isdir(TRAIN_OUT) else os.listdir(TRAIN_OUT)[:5])
```

```
Using nested folder /content/datasets/images as dataset root
Detected classes (if any): ['test', 'train', 'val']
Preserving class subfolders. Copying up to MAX_PER_CLASS per class until MAX_IMAGES total reached.
Copied total 250 images into subset /content/datasets/african_small
Train sample files: ['train', 'test']
```

```
# CELL 3 - Create data_custom.yaml using the real dataset yaml
import os, yaml

SUBSET = '/content/datasets/african_small'
train_path = os.path.join(SUBSET, 'images', 'train')
val_path   = os.path.join(SUBSET, 'images', 'val')

# Use the real yaml shipped with the dataset if available
real_yaml = '/content/datasets/african-wildlife.yaml'
if os.path.exists(real_yaml):
    with open(real_yaml) as f:
        src = yaml.safe_load(f)
    names = src.get('names', [])
    print("Loaded class names from dataset yaml:", names)
```

```
else:
    # Fallback: known african-wildlife classes
    names = ['buffalo', 'elephant', 'rhino', 'zebra']
    print("Using hardcoded class names:", names)

data = {
    'train': train_path,
    'val':   val_path,
    'nc':    len(names),
    'names': names
}
yaml_path = os.path.join(SUBSET, 'data_custom.yaml')
with open(yaml_path, 'w') as f:
    yaml.dump(data, f)

print("Wrote dataset YAML:", yaml_path)
print(data)
```

```
Loaded class names from dataset yaml: {0: 'buffalo', 1: 'elephant', 2: 'rhino', 3: 'zebra'}
Wrote dataset YAML: /content/datasets/african_small/data_custom.yaml
{'train': '/content/datasets/african_small/images/train', 'val': '/content/datasets/african_small/images/val', 'nc': 4, 'nam
```

```
# CELL 4 - Copy real YOLO labels (preserving class subfolders) OR generate pseudo-labels
import os, glob, shutil
from pathlib import Path
from PIL import Image  # import FIRST

SUBSET = Path('/content/datasets/african_small')

# Mirror the image subfolder structure in labels/
for split in ['train', 'val']:
    img_root = SUBSET / 'images' / split
    lbl_root = SUBSET / 'labels' / split
    shutil.rmtree(lbl_root, ignore_errors=True)

    for img_path in sorted(img_root.rglob('*.*')):
        if img_path.suffix.lower() not in ('.jpg', '.jpeg', '.png', '.bmp'):
            continue
        # Preserve subfolder structure (e.g., labels/train/zebra/)
        rel = img_path.relative_to(img_root)
        lbl_path = lbl_root / rel.parent / (img_path.stem + '.txt')
        lbl_path.parent.mkdir(parents=True, exist_ok=True)

        # Try to find a matching real label anywhere under /content/datasets
        # (excluding our subset) by filename stem
        matches = [
            p for p in glob.glob(f'/content/datasets/**/{img_path.stem}.txt', recursive=True)
            if 'african_small' not in p
        ]
        if matches:
            shutil.copy2(matches[0], lbl_path)
        else:
            lbl_path.write_text('')  # placeholder

# Count non-empty labels copied
real_labels = [
    p for p in SUBSET.rglob('labels/**/*.txt') if os.path.getsize(p) > 0
]
print(f"Non-empty label files found: {len(real_labels)}")

# ── Pseudo-label only images that still have empty labels ──────────────
GENERATE_PSEUDO = True
CONF = 0.35

# Load yaml to get nc for class remapping
import yaml
with open(str(SUBSET / 'data_custom.yaml')) as f:
    data_cfg = yaml.safe_load(f)
NC = data_cfg['nc']

if GENERATE_PSEUDO:
    empty_labels = [
        p for p in SUBSET.rglob('labels/**/*.txt') if os.path.getsize(p) == 0
    ]
    if not empty_labels:
        print("All labels already populated — skipping pseudo-labeling.")
    else:
        print(f"Generating pseudo-labels for {len(empty_labels)} unlabelled images...")
        from ultralytics import YOLO
        model = YOLO('yolov8n.pt')
        total_boxes = 0
```

```
        def preds_to_yolo_txt(res, img_path, out_txt_path, conf_thr, nc):
            """Write YOLO-format label; remap class IDs to [0, nc-1]."""
            W, H = Image.open(img_path).size
            lines = []
            boxes = getattr(res, 'boxes', None) or []
            for b in boxes:
                conf = float(b.conf.cpu().numpy()[0])
                if conf < conf_thr:
                    continue
                raw_cls = int(b.cls.cpu().numpy()[0])
                cls = raw_cls % nc          # remap COCO id → valid range
                xyxy = b.xyxy.cpu().numpy()[0]
                x1, y1, x2, y2 = xyxy
                xc = (x1 + x2) / (2 * W)
                yc = (y1 + y2) / (2 * H)
                bw = (x2 - x1) / W
                bh = (y2 - y1) / H
                lines.append(f"{cls} {xc:.6f} {yc:.6f} {bw:.6f} {bh:.6f}")
            Path(out_txt_path).write_text("\n".join(lines))
            return len(lines)

        for lbl_path in empty_labels:
            # Reconstruct matching image path
            lbl_path = Path(lbl_path)
            rel = lbl_path.relative_to(SUBSET / 'labels')   # e.g. train/zebra/img.txt
            img_path = SUBSET / 'images' / rel.parent / (lbl_path.stem + '.jpg')
            if not img_path.exists():
                # Try other extensions
                for ext in ('.png', '.jpeg', '.bmp'):
                    candidate = img_path.with_suffix(ext)
                    if candidate.exists():
                        img_path = candidate
                        break
            if not img_path.exists():
                continue
            res = model.predict(source=str(img_path), imgsz=640, conf=CONF, verbose=False)[0]
            n = preds_to_yolo_txt(res, str(img_path), str(lbl_path), CONF, NC)
            total_boxes += n

        print(f"Pseudo-labeling done. Total boxes written: {total_boxes}")
else:
    print("Skipping pseudo-label generation.")
```

```
Non-empty label files found: 250
All labels already populated — skipping pseudo-labeling.
```

```
# CELL 5 - Verify images & labels with subfolder-aware counts
import yaml, glob, os

yaml_path = '/content/datasets/african_small/data_custom.yaml'
with open(yaml_path) as f:
    data = yaml.safe_load(f)
print("YAML:", data)

def count_images(path):
    return len([p for p in glob.glob(os.path.join(path,'**','*.*'), recursive=True)
                if p.lower().endswith(('.jpg','.jpeg','.png','.bmp'))])

def count_labels(path, non_empty=False):
    txts = glob.glob(os.path.join(path,'**','*.txt'), recursive=True)
    if non_empty:
        return sum(1 for t in txts if os.path.getsize(t) > 0)
    return len(txts)

print(f"\nTrain images : {count_images(data['train'])}")
print(f"Val   images : {count_images(data['val'])}")

lbl_train = '/content/datasets/african_small/labels/train'
lbl_val   = '/content/datasets/african_small/labels/val'
print(f"\nTrain labels (total)     : {count_labels(lbl_train)}")
print(f"Train labels (non-empty) : {count_labels(lbl_train, non_empty=True)}")
print(f"Val   labels (total)     : {count_labels(lbl_val)}")
print(f"Val   labels (non-empty) : {count_labels(lbl_val, non_empty=True)}")

# Show a non-empty example
for l in glob.glob(os.path.join(lbl_val,'**','*.txt'), recursive=True):
    if os.path.getsize(l) > 0:
        print(f"\nExample label ({l}):\n{open(l).read()}")
        break

print("\nIf non-empty label counts > 0, training should work. Re-run CELL 6.")
```

```
YAML: {'names': {0: 'buffalo', 1: 'elephant', 2: 'rhino', 3: 'zebra'}, 'nc': 4, 'train': '/content/datasets/african_small/im

Train images : 210
Val   images : 40

Train labels (total)     : 210
Train labels (non-empty) : 210
Val   labels (total)     : 40
Val   labels (non-empty) : 40

Example label (/content/datasets/african_small/labels/val/test/4 (179).txt):
3 0.542969 0.629167 0.235937 0.595833
3 0.654687 0.446875 0.381250 0.477083
3 0.385937 0.416667 0.315625 0.516667


If non-empty label counts > 0, training should work. Re-run CELL 6.
```

```
import torch
import ultralytics.nn.tasks as ult_tasks
import ultralytics.nn.modules as ult_modules

# PyTorch 2.6 changed weights_only=True by default, which blocks Ultralytics globals.
# We need to allowlist the required Ultralytics classes.

safe_classes = [
    ult_tasks.DetectionModel,
    ult_tasks.SegmentationModel,
    ult_tasks.ClassificationModel,
    ult_tasks.PoseModel,
]

# Add each class to PyTorch's safe globals list
for cls in safe_classes:
    try:
        torch.serialization.add_safe_globals([cls])
    except Exception as e:
        print(f"Could not add {cls}: {e}")

# Also monkey-patch torch.load to force weights_only=False for .pt files
# (safe since we are loading from ultralytics official weights)
_original_torch_load = torch.load

def _patched_torch_load(f, *args, **kwargs):
    kwargs.setdefault('weights_only', False)
    return _original_torch_load(f, *args, **kwargs)

torch.load = _patched_torch_load

print("PyTorch version   :", torch.__version__)
print("Torch.load patched: weights_only now defaults to False")
print("Safe globals registered for Ultralytics model classes.")
```

```
PyTorch version   : 2.10.0+cpu
Torch.load patched: weights_only now defaults to False
Safe globals registered for Ultralytics model classes.
```

```
# CELL 6 - YOLOv8 Training with validation image saving
from ultralytics import YOLO
import shutil, glob, os

data_yaml_local = '/content/datasets/african_small/data_custom.yaml'
print("Training YOLOv8 on:", data_yaml_local)

model_v8 = YOLO("yolov8n.pt")
results_v8 = model_v8.train(
    data=data_yaml_local,
    epochs=6,
    imgsz=640,
    batch=8,
    workers=2,
    patience=0,
    save=True,
    name='yolov8_african'
)

# Save best weights
v8_candidates = glob.glob('runs/detect/yolov8_african*/weights/best.pt')
if v8_candidates:
    shutil.copy(v8_candidates[-1], '/content/yolov8_best.pt')
    print("Saved: /content/yolov8_best.pt")
else:
```

```
    print("WARNING: yolov8_best.pt not found")

# Run validation and save images
model_v8_best = YOLO('/content/yolov8_best.pt')
val_results_v8 = model_v8_best.val(
    data=data_yaml_local,
    imgsz=640,
    save=True,
    name='yolov8_val'
)

# Copy validation images to organized output folder
os.makedirs('/content/outputs/yolov8/val_images', exist_ok=True)
for img in glob.glob('runs/detect/yolov8_val*/**/*.jpg', recursive=True):
    shutil.copy(img, '/content/outputs/yolov8/val_images/')
for img in glob.glob('runs/detect/yolov8_val*/**/*.png', recursive=True):
    shutil.copy(img, '/content/outputs/yolov8/val_images/')

print(f"YOLOv8 mAP50: {val_results_v8.box.map50:.4f}")
print(f"YOLOv8 mAP50-95: {val_results_v8.box.map:.4f}")
print("Validation images saved to /content/outputs/yolov8/val_images/")
```

```
Training YOLOv8 on: /content/datasets/african_small/data_custom.yaml
New https://pypi.org/project/ultralytics/8.4.17 available 😃 Update with 'pip install -U ultralytics'
Ultralytics YOLOv8.1.23 🚀 Python-3.12.12 torch-2.10.0+cpu CPU (Intel Xeon 2.20GHz)
engine/trainer: task=detect, mode=train, model=yolov8n.pt, data=/content/datasets/african_small/data_custom.yaml, epochs=6
Overriding model.yaml nc=80 with nc=4

                   from  n    params  module                                       arguments
  0                  -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                  -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                  -1  1      7360  ultralytics.nn.modules.block.C2f             [32, 32, 1, True]
  3                  -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
  4                  -1  2     49664  ultralytics.nn.modules.block.C2f             [64, 64, 2, True]
  5                  -1  1     73984  ultralytics.nn.modules.conv.Conv             [64, 128, 3, 2]
  6                  -1  2    197632  ultralytics.nn.modules.block.C2f             [128, 128, 2, True]
  7                  -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                  -1  1    460288  ultralytics.nn.modules.block.C2f             [256, 256, 1, True]
  9                  -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5]
 10                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 11             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 12                  -1  1    148224  ultralytics.nn.modules.block.C2f             [384, 128, 1]
 13                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 14             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 15                  -1  1     37248  ultralytics.nn.modules.block.C2f             [192, 64, 1]
 16                  -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 17            [-1, 12]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 18                  -1  1    123648  ultralytics.nn.modules.block.C2f             [192, 128, 1]
 19                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 20             [-1, 9]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 21                  -1  1    493056  ultralytics.nn.modules.block.C2f             [384, 256, 1]
 22        [15, 18, 21]  1    752092  ultralytics.nn.modules.head.Detect           [4, [64, 128, 256]]
Model summary: 225 layers, 3011628 parameters, 3011612 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
TensorBoard: Start with 'tensorboard --logdir runs/detect/yolov8_african', view at http://localhost:6006/
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 1
wandb: You chose 'Create a W&B account'
wandb: Create an account here: https://wandb.ai/authorize?signup=true&ref=models
wandb: After creating your account, create a new API key and store it securely.
wandb: Paste your API key and hit enter: ··········
wandb: ERROR Invalid API key: API key must have 40+ characters, has 3.
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 3
wandb: You chose "Don't visualize my results"
wandb: Using W&B in offline mode.
wandb: W&B API key is configured. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.25.0
W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to enable cloud syncing.
Run data is saved locally in /content/wandb/offline-run-20260226_172826-w44dan5q
Freezing layer 'model.22.dfl.conv.weight'
/usr/local/lib/python3.12/dist-packages/ultralytics/engine/trainer.py:271: FutureWarning: `torch.cuda.amp.GradScaler(args.
  self.scaler = torch.cuda.amp.GradScaler(enabled=self.amp)
train: Scanning /content/datasets/african_small/labels/train/test... 210 images, 0 backgrounds, 0 corrupt: 100%|██████████

albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_av
/usr/local/lib/python3.12/dist-packages/ultralytics/data/augment.py:846: UserWarning: Argument(s) 'quality_lower' are not
  A.ImageCompression(quality_lower=75, p=0.0),
/usr/local/lib/python3.12/dist-packages/albumentations/core/composition.py:331: UserWarning: Got processor for bboxes, but
  self._set_keys()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as t
  super().__init__(loader)
val: Scanning /content/datasets/african_small/labels/val/test... 40 images, 0 backgrounds, 0 corrupt: 100%|██████████| 40/

Plotting labels to runs/detect/yolov8_african/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'm
optimizer: AdamW(lr=0.00125, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(de
TensorBoard: model graph visualization added ✅
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to runs/detect/yolov8_african
Starting training for 6 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
        1/6         0G     0.7645      2.819       1.22         10        640: 100%|██████████| 27/27 [03:06<00:00,  6.91s
                  Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 3/3 [00:09<00:0


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
        2/6         0G     0.8189      2.088      1.264          6        640: 100%|██████████| 27/27 [02:49<00:00,  6.27s
                  Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 3/3 [00:09<00:0


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
        3/6         0G     0.8306      1.853      1.233          9        640: 100%|██████████| 27/27 [02:50<00:00,  6.33s
```

```
# CELL 7 - YOLOv5 Training
import os, shutil, glob, subprocess

data_yaml_local = '/content/datasets/african_small/data_custom.yaml'

# Clone YOLOv5 if not already present
if not os.path.exists('/content/yolov5'):
    subprocess.run(['git', 'clone', '-q', 'https://github.com/ultralytics/yolov5.git', '/content/yolov5'])
    subprocess.run(['pip', 'install', '-q', '-r', '/content/yolov5/requirements.txt'])

os.makedirs('/content/outputs/yolov5/val_images', exist_ok=True)

# Train YOLOv5n
%cd /content/yolov5
!python train.py \
    --img 640 \
    --batch 8 \
    --epochs 6 \
    --data {data_yaml_local} \
    --weights yolov5n.pt \
    --name yolov5_african \
    --patience 0 \
    --save-period 1

%cd /content

# Save best weights
v5_candidates = glob.glob('/content/yolov5/runs/train/yolov5_african*/weights/best.pt')
if v5_candidates:
    shutil.copy(v5_candidates[-1], '/content/yolov5_best.pt')
    print("Saved: /content/yolov5_best.pt")
else:
    print("WARNING: yolov5 best.pt not found")

# Run validation and save images
%cd /content/yolov5
!python val.py \
    --img 640 \
    --data {data_yaml_local} \
    --weights /content/yolov5_best.pt \
    --name yolov5_val \
    --save-txt \
    --save-conf

%cd /content

# Copy validation images
for img in glob.glob('/content/yolov5/runs/val/yolov5_val*/**/*.jpg', recursive=True):
    shutil.copy(img, '/content/outputs/yolov5/val_images/')
for img in glob.glob('/content/yolov5/runs/val/yolov5_val*/**/*.png', recursive=True):
    shutil.copy(img, '/content/outputs/yolov5/val_images/')

print("YOLOv5 validation images saved to /content/outputs/yolov5/val_images/")

# Parse YOLOv5 metrics from results.csv
import pandas as pd
results_csv = glob.glob('/content/yolov5/runs/train/yolov5_african*/results.csv')
if results_csv:
    df = pd.read_csv(results_csv[-1])
    df.columns = df.columns.str.strip()
    last = df.iloc[-1]
    v5_map50 = float(last['metrics/mAP50(B)']) if 'metrics/mAP50(B)' in df.columns else 0.0
    v5_map5095 = float(last['metrics/mAP50-95(B)']) if 'metrics/mAP50-95(B)' in df.columns else 0.0
    print(f"YOLOv5 mAP50: {v5_map50:.4f}")
    print(f"YOLOv5 mAP50-95: {v5_map5095:.4f}")
else:
    v5_map50, v5_map5095 = 0.0, 0.0
    print("Could not find YOLOv5 results.csv")


# Add to end of Cell 7 — save weights outside yolov5 folder immediately
import glob, shutil
v5_candidates = glob.glob('/content/yolov5/runs/train/yolov5_african*/weights/best.pt')
if v5_candidates:
    shutil.copy(v5_candidates[-1], '/content/yolov5_best.pt')
    print("Saved: /content/yolov5_best.pt")
else:
    print("WARNING: yolov5 best.pt not found — check Cell 7 training completed.")
```

```
YOLOv8 mAP50: 0.2155
YOLOv8 mAP50-95: 0.1530
Validation images saved to /content/outputs/yolov8/val_images/
```

```
       5/5         0G    0.05483    0.02823    0.03384         33        640:  85% 23/27 [01:37<00:16,  4.20s/it]/content
  with torch.cuda.amp.autocast(amp):
       5/5         0G    0.05538    0.02823     0.0339         32        640:  89% 24/27 [01:41<00:12,  4.12s/it]/content
  with torch.cuda.amp.autocast(amp):
       5/5         0G    0.05546    0.02811    0.03393         27        640:  93% 25/27 [01:46<00:08,  4.42s/it]/content
  with torch.cuda.amp.autocast(amp):
       5/5         0G    0.05583    0.02878    0.03389         45        640:  96% 26/27 [01:50<00:04,  4.30s/it]/content
  with torch.cuda.amp.autocast(amp):
       5/5         0G    0.05552    0.02838    0.03351          7        640: 100% 27/27 [01:51<00:00,  4.12s/it]
                 Class     Images  Instances          P          R      mAP50   mAP50-95: 100% 3/3 [00:06<00:00,  2.16s/it
                   all         40         72     0.0226      0.962      0.169     0.0735

6 epochs completed in 0.201 hours.
Traceback (most recent call last):
  File "/content/yolov5/train.py", line 987, in <module>
    main(opt)
  File "/content/yolov5/train.py", line 688, in main
    train(opt.hyp, opt, device, callbacks)
  File "/content/yolov5/train.py", line 519, in train
    strip_optimizer(f)  # strip optimizers
    ^^^^^^^^^^^^^^^^^^
  File "/content/yolov5/utils/general.py", line 1124, in strip_optimizer
    x = torch_load(f, map_location=torch.device("cpu"))
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/torch/serialization.py", line 1548, in load
    raise pickle.UnpicklingError(_get_wo_message(str(e))) from None
_pickle.UnpicklingError: Weights only load failed. This file can still be loaded, to do so you have two options, do those
        (1) In PyTorch 2.6, we changed the default value of the `weights_only` argument in `torch.load` from `False` to `T
        (2) Alternatively, to load with `weights_only=True` please check the recommended steps in the following error mess
        WeightsUnpickler error: Unsupported global: GLOBAL numpy._core.multiarray._reconstruct was not an allowed global b

Check the documentation of torch.load to learn more about types accepted by default with weights_only https://pytorch.org/
/content
Saved: /content/yolov5_best.pt
/content/yolov5
val: data=/content/datasets/african_small/data_custom.yaml, weights=['/content/yolov5_best.pt'], batch_size=32, imgsz=640,
YOLOv5 🚀 v7.0-460-g3fb11111 Python-3.12.12 torch-2.10.0+cpu CPU

Fusing layers...
Model summary: 157 layers, 1764577 parameters, 0 gradients, 4.1 GFLOPs
val: Scanning /content/datasets/african_small/labels/val/test.cache... 40 images, 0 backgrounds, 0 corrupt: 100% 40/40 [00
                 Class     Images  Instances          P          R      mAP50   mAP50-95: 100% 2/2 [00:12<00:00,  6.22s/it
                   all         40         72     0.0226      0.962       0.17     0.0732
              elephant         40          1   0.000312          1     0.0249     0.0152
                 rhino         40          1    0.00039          1     0.0142    0.00646
                 zebra         40         70     0.0672      0.886       0.47      0.198
Speed: 16.5ms pre-process, 215.3ms inference, 6.4ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs/val/yolov5_val
40 labels saved to runs/val/yolov5_val/labels
/content
YOLOv5 validation images saved to /content/outputs/yolov5/val_images/
YOLOv5 mAP50: 0.0000
YOLOv5 mAP50-95: 0.0000
Saved: /content/yolov5_best.pt
```

```
# CELL 8 - Ensemble Learning using Weighted Boxes Fusion (WBF)
import os, glob, shutil, sys
import numpy as np
from PIL import Image
from ensemble_boxes import weighted_boxes_fusion
import torch
import yaml

os.makedirs('/content/outputs/ensemble/val_images', exist_ok=True)
os.makedirs('/content/outputs/ensemble/test_images', exist_ok=True)

# ── Step 1: Back up weights BEFORE re-cloning ──────────────────────
# The re-clone deletes /content/yolov5 including any trained weights inside it.
# So we copy them out first.
v5_weight_src = glob.glob('/content/yolov5/runs/train/**/weights/best.pt', recursive=True)
if not os.path.exists('/content/yolov5_best.pt'):
    if v5_weight_src:
        shutil.copy(v5_weight_src[-1], '/content/yolov5_best.pt')
        print(f"Backed up YOLOv5 weights to /content/yolov5_best.pt")
    else:
        raise FileNotFoundError(
            "No YOLOv5 weights found. "
            "Please re-run Cell 7 (YOLOv5 training) before running this cell."
        )
else:
    print("YOLOv5 weights already at /content/yolov5_best.pt — skipping backup.")

# ── Step 2: Re-clone YOLOv5 fresh ──────────────────────────────────
print("Re-cloning YOLOv5...")
shutil.rmtree('/content/yolov5', ignore_errors=True)
os.system('git clone -q https://github.com/ultralytics/yolov5.git /content/yolov5')
```

```
# Filter ultralytics out of requirements to prevent version downgrade
with open('/content/yolov5/requirements.txt', 'r') as f:
    reqs = f.readlines()
filtered = [r for r in reqs if 'ultralytics' not in r.lower()]
with open('/content/yolov5/requirements_filtered.txt', 'w') as f:
    f.writelines(filtered)
os.system('pip install -q -r /content/yolov5/requirements_filtered.txt')
os.system('pip install -q -U "ultralytics>=8.3.0"')
print("YOLOv5 re-cloned and deps installed.")

# ── Step 3: Patch general.py ──────────────────────────────────────────
general_py = '/content/yolov5/utils/general.py'
with open(general_py, 'r') as f:
    lines = f.readlines()

broken  = 'from ultralytics.utils.patches import torch_load'
patched = 'torch_load = torch.load  # patched: torch_load removed from ultralytics\n'

new_lines = []
patch_applied = False
for line in lines:
    if line.strip() == broken:
        new_lines.append(patched)
        patch_applied = True
    else:
        new_lines.append(line)

with open(general_py, 'w') as f:
    f.writelines(new_lines)
print(f"Patch applied: {patch_applied}")

# ── Step 4: Clear stale module cache ──────────────────────────────────
to_delete = [k for k in sys.modules.keys()
             if any(x in k for x in [
                 'models', 'utils.general', 'utils.augment',
                 'utils.torch_utils', 'utils.dataloaders',
                 'utils.loss', 'utils.metrics', 'utils.plots',
                 'ultralytics'
             ])]
for k in to_delete:
    del sys.modules[k]
print(f"Cleared {len(to_delete)} cached modules.")

# ── Step 5: Add yolov5 to path and import ─────────────────────────────
if '/content/yolov5' in sys.path:
    sys.path.remove('/content/yolov5')
sys.path.insert(0, '/content/yolov5')

from models.common import DetectMultiBackend
from utils.general import non_max_suppression, scale_boxes
from utils.augmentations import letterbox
print("YOLOv5 imports successful.")

# ── Step 6: Load both models ──────────────────────────────────────────
from ultralytics import YOLO
model_v8 = YOLO('/content/yolov8_best.pt')
print("YOLOv8 loaded.")

def load_yolov5_local(weights_path, device='cpu'):
    model = DetectMultiBackend(weights_path, device=torch.device(device))
    model.eval()
    return model

model_v5_local = load_yolov5_local('/content/yolov5_best.pt')
print("YOLOv5 loaded.")

# ── Step 7: Inference helpers ─────────────────────────────────────────
def yolov5_predict(model, img_path, conf_thr=0.25, iou_thr=0.45, imgsz=640):
    img0 = np.array(Image.open(img_path).convert('RGB'))
    H0, W0 = img0.shape[:2]
    img = letterbox(img0, imgsz, stride=int(model.stride), auto=True)[0]
    img = img.transpose((2, 0, 1))[::-1]
    img = np.ascontiguousarray(img)
    img_tensor = torch.from_numpy(img).float() / 255.0
    img_tensor = img_tensor.unsqueeze(0)
    with torch.no_grad():
        pred = model(img_tensor)
        pred = non_max_suppression(pred, conf_thr, iou_thr)
    boxes, scores, labels = [], [], []
    if pred[0] is not None and len(pred[0]):
        det = pred[0].clone()
        det[:, :4] = scale_boxes(img_tensor.shape[2:], det[:, :4], img0.shape).round()
```

```
        for *xyxy, conf, cls in det.tolist():
            x1, y1, x2, y2 = xyxy
            boxes.append([max(0,x1/W0), max(0,y1/H0), min(1,x2/W0), min(1,y2/H0)])
            scores.append(float(conf))
            labels.append(int(cls))
    return boxes, scores, labels

# ── Step 8: Ensemble config ──────────────────────────────────────────
IOU_THR  = 0.5
SKIP_BOX = 0.0001
CONF_THR = 0.25
WEIGHTS  = [1, 1]

with open('/content/datasets/african_small/data_custom.yaml') as f:
    cfg = yaml.safe_load(f)
class_names = cfg['names']

def run_ensemble_on_image(img_path):
    img  = Image.open(img_path).convert('RGB')
    W, H = img.size
    all_boxes, all_scores, all_labels = [], [], []

    # YOLOv8 predictions
    res_v8 = model_v8.predict(source=img_path, imgsz=640, conf=CONF_THR, verbose=False)[0]
    b8, s8, l8 = [], [], []
    if res_v8.boxes is not None:
        for b in res_v8.boxes:
            x1,y1,x2,y2 = b.xyxy.cpu().numpy()[0]
            b8.append([max(0,x1/W), max(0,y1/H), min(1,x2/W), min(1,y2/H)])
            s8.append(float(b.conf.cpu().numpy()[0]))
            l8.append(int(b.cls.cpu().numpy()[0]))
    all_boxes.append(b8 if b8 else [[0,0,0,0]])
    all_scores.append(s8 if s8 else [0.0])
    all_labels.append(l8 if l8 else [0])

    # YOLOv5 predictions
    b5, s5, l5 = yolov5_predict(model_v5_local, img_path, CONF_THR)
    all_boxes.append(b5 if b5 else [[0,0,0,0]])
    all_scores.append(s5 if s5 else [0.0])
    all_labels.append(l5 if l5 else [0])

    fused_boxes, fused_scores, fused_labels = weighted_boxes_fusion(
        all_boxes, all_scores, all_labels,
        weights=WEIGHTS, iou_thr=IOU_THR, skip_box_thr=SKIP_BOX
    )
    return fused_boxes, fused_scores, fused_labels, W, H


def draw_and_save(img_path, fused_boxes, fused_scores,
                  fused_labels, W, H, out_path, class_names):
    import cv2
    img_cv = cv2.imread(img_path)
    if img_cv is None:
        img_cv = cv2.cvtColor(
            np.array(Image.open(img_path).convert('RGB')), cv2.COLOR_RGB2BGR)
    for box, score, label in zip(fused_boxes, fused_scores, fused_labels):
        x1=int(box[0]*W); y1=int(box[1]*H)
        x2=int(box[2]*W); y2=int(box[3]*H)
        cls_name = (class_names[int(label)]
                    if int(label) < len(class_names) else str(int(label)))
        cv2.rectangle(img_cv, (x1,y1), (x2,y2), (0,255,0), 2)
        cv2.putText(img_cv, f"{cls_name} {score:.2f}",
                    (x1, max(y1-5, 10)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)
    cv2.imwrite(out_path, img_cv)


# ── Step 9: Run ensemble on VAL images ──────────────────────────────
val_images = glob.glob(
    '/content/datasets/african_small/images/val/**/*.*', recursive=True)
val_images = [p for p in val_images
              if p.lower().endswith(('.jpg','.jpeg','.png','.bmp'))]
print(f"Running ensemble on {len(val_images)} validation images...")

processed, skipped = 0, 0
for img_path in val_images:
    try:
        fused_boxes, fused_scores, fused_labels, W, H = run_ensemble_on_image(img_path)
        out_path = f'/content/outputs/ensemble/val_images/{os.path.basename(img_path)}'
        draw_and_save(img_path, fused_boxes, fused_scores,
                      fused_labels, W, H, out_path, class_names)
        processed += 1
```

```
    except Exception as e:
        print(f"  Skipped {os.path.basename(img_path)}: {e}")
        skipped += 1

print(f"\nDone. {processed} saved, {skipped} skipped.")
print("Ensemble val images: /content/outputs/ensemble/val_images/")
```

```
YOLOv5 weights already at /content/yolov5_best.pt — skipping backup.
Re-cloning YOLOv5...
YOLOv5 re-cloned and deps installed.
Patch applied: True
Cleared 113 cached modules.
YOLOv5 imports successful.
YOLOv8 loaded.
Fusing layers...
Model summary: 157 layers, 1764577 parameters, 0 gradients, 4.1 GFLOPs
YOLOv5 loaded.
Running ensemble on 40 validation images...

Done. 40 saved, 0 skipped.
Ensemble val images: /content/outputs/ensemble/val_images/
```

```
# CELL 9 - Run all 3 models on TEST images and save outputs
import os, glob, shutil
import torch
from ultralytics import YOLO

os.makedirs('/content/outputs/yolov8/test_images', exist_ok=True)
os.makedirs('/content/outputs/yolov5/test_images', exist_ok=True)
os.makedirs('/content/outputs/ensemble/test_images', exist_ok=True)

# ── Find test images ──────────────────────────────────────────────
test_images = glob.glob(
    '/content/datasets/african_small/images/test/**/*.*', recursive=True)
test_images = [p for p in test_images
               if p.lower().endswith(('.jpg','.jpeg','.png','.bmp'))]

# Fallback to val if no test split
if not test_images:
    print("No test split found — using val images as test set.")
    test_images = glob.glob(
        '/content/datasets/african_small/images/val/**/*.*', recursive=True)
    test_images = [p for p in test_images
                   if p.lower().endswith(('.jpg','.jpeg','.png','.bmp'))]

print(f"Found {len(test_images)} test images.")

# ── YOLOv8 test inference ─────────────────────────────────────────
print("Running YOLOv8 on test images...")
model_v8 = YOLO('/content/yolov8_best.pt')
results_v8_test = model_v8.predict(
    source=test_images,
    imgsz=640,
    conf=0.25,
    save=True,
    name='yolov8_test'
)
for img in glob.glob('runs/detect/yolov8_test*/**/*.jpg', recursive=True):
    shutil.copy(img, '/content/outputs/yolov8/test_images/')
for img in glob.glob('runs/detect/yolov8_test*/**/*.png', recursive=True):
    shutil.copy(img, '/content/outputs/yolov8/test_images/')
print(f"YOLOv8 test images saved: /content/outputs/yolov8/test_images/")

# ── YOLOv5 test inference (using local model from Cell 8) ─────────
# model_v5_local is already loaded in Cell 8 — reuse it directly
print("Running YOLOv5 on test images...")
import cv2
import numpy as np
from PIL import Image
import yaml

with open('/content/datasets/african_small/data_custom.yaml') as f:
    cfg = yaml.safe_load(f)
class_names = cfg['names']

for img_path in test_images:
    try:
        b5, s5, l5 = yolov5_predict(model_v5_local, img_path, conf_thr=0.25)
        img_cv = cv2.imread(img_path)
        if img_cv is None:
            img_cv = cv2.cvtColor(
                np.array(Image.open(img_path).convert('RGB')), cv2.COLOR_RGB2BGR)
        h, w = img_cv.shape[:2]
```

```
            for box, score, label in zip(b5, s5, l5):
                x1=int(box[0]*w); y1=int(box[1]*h)
                x2=int(box[2]*w); y2=int(box[3]*h)
                cls_name = class_names[label] if label < len(class_names) else str(label)
                cv2.rectangle(img_cv, (x1,y1), (x2,y2), (255,128,0), 2)
                cv2.putText(img_cv, f"{cls_name} {score:.2f}",
                            (x1, max(y1-5,10)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,128,0), 2)
            out_path = f'/content/outputs/yolov5/test_images/{os.path.basename(img_path)}'
            cv2.imwrite(out_path, img_cv)
        except Exception as e:
            print(f"  YOLOv5 skipped {os.path.basename(img_path)}: {e}")

    print(f"YOLOv5 test images saved: /content/outputs/yolov5/test_images/")

    # ── Ensemble test inference (reuse run_ensemble_on_image from Cell 8) ──────
    print("Running Ensemble on test images...")
    processed, skipped = 0, 0
    for img_path in test_images:
        try:
            fused_boxes, fused_scores, fused_labels, W, H = run_ensemble_on_image(img_path)
            out_path = f'/content/outputs/ensemble/test_images/{os.path.basename(img_path)}'
            draw_and_save(img_path, fused_boxes, fused_scores,
                          fused_labels, W, H, out_path, class_names)
            processed += 1
        except Exception as e:
            print(f"  Ensemble skipped {os.path.basename(img_path)}: {e}")
            skipped += 1

    print(f"Ensemble test images saved: /content/outputs/ensemble/test_images/ ({processed} images)")

    # ── Summary ──────────────────────────────────────────────────────────────
    print("\n── Test inference complete ──")
    for folder, label in [
        ('/content/outputs/yolov8/test_images', 'YOLOv8'),
        ('/content/outputs/yolov5/test_images', 'YOLOv5'),
        ('/content/outputs/ensemble/test_images', 'Ensemble'),
    ]:
        count = len([f for f in glob.glob(os.path.join(folder, '*'))
                     if f.lower().endswith(('.jpg','.png','.jpeg'))])
        print(f"  {label:10s}: {count} images in {folder}")
```